

V1 실행을 위해 마스터가 작업할 작업확인서

이 문서는 V0 상태의 **phoenix pages**를 V1으로 업그레이드하기 전에 마스터가 직접 준비하고 확인해야 할 항목을 정리한 작업확인서이다.

수강생은 이 문서를 보면서 Supabase, Google OAuth, Vercel 환경변수를 준비하고, 코텍스에게 V1 작업을 맡기기 전에 체크리스트를 완료한다.

1. V1에서 마스터가 하는 일

코드는 코텍스가 수정할 수 있다.

하지만 아래 작업은 마스터가 직접 해야 한다.

1. Supabase 프로젝트 준비
2. Supabase URL과 key 확인
3. Supabase Storage bucket 생성
4. Google OAuth Client 생성
5. Supabase Auth Google Provider 설정
6. Supabase URL Configuration 설정
7. 슈퍼 관리자 이메일 결정
8. Vercel 환경변수 입력
9. 코텍스가 제공한 SQL 실행
10. V1 배포 후 로그인과 저장 테스트

2. 준비 전 확인

작업 전 아래 계정이 준비되어 있어야 한다.

- [] GitHub 계정이 있다.
- [] Vercel 계정이 있다.
- [] Supabase 계정이 있다.
- [] Google 계정이 있다.
- [] Google Cloud Console에 접속할 수 있다.
- [] **phoenix pages** V0 저장소를 가지고 있다.
- [] Vercel에 V0 프로젝트가 배포되어 있다.

현재 V0 저장소:

<https://github.com/phoenixai-sw/phoenix-detail-page.git>

3. Supabase 프로젝트 확인

3.1 Supabase Dashboard 접속

1. 브라우저에서 Supabase Dashboard에 접속한다.

<https://supabase.com/dashboard>

2. 로그인한다.
3. V1에 사용할 프로젝트를 선택한다.

프로젝트가 없다면:

1. **New project**를 누른다.
2. Organization을 선택한다.
3. Project name을 입력한다.

추천:

phoenix-pages

4. Database Password를 만든다.
5. Region을 선택한다.
6. **Create new project**를 누른다.

체크:

- [] Supabase 프로젝트가 준비됐다.
- [] 프로젝트 Dashboard에 접속 가능하다.
- [] 왼쪽 메뉴에 **Authentication, Storage, Table Editor, SQL Editor, Project Settings**가 보인다.

4. Supabase Project URL 확인

4.1 메뉴 이동

1. Supabase 프로젝트를 연다.
2. 왼쪽 메뉴에서 **Project Settings**를 누른다.
3. **API Keys** 또는 **API** 메뉴를 연다.
4. **Project URL**을 찾는다.

값 모양:

<https://xxxxxxxxxxxxxxxxxxxxx.supabase.co>

기록:

NEXT_PUBLIC_SUPABASE_URL=

체크:

- [] Project URL을 복사했다.
- [] NEXT_PUBLIC_SUPABASE_URL 이름으로 기록했다.

5. Supabase 공개용 key 확인

5.1 메뉴 이동

1. Supabase 프로젝트를 연다.
2. **Project Settings**로 이동한다.
3. **API Keys** 메뉴를 연다.
4. **Publishable key** 또는 **anon public key**를 찾는다.

새 키 체계:

sb_publishable_...

레거시 anon key:

eyJhbGciOi...

기록:

NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=

또는:

NEXT_PUBLIC_SUPABASE_ANON_KEY=

체크:

- [] 공개용 key를 복사했다.
- [] publishable key인지 anon key인지 구분했다.
- [] 공개용 key에만 NEXT_PUBLIC_ 접두사를 붙였다.

6. Supabase 서버용 secret/service role key 확인

이 값은 매우 중요하다.

절대 공개하면 안 된다.

6.1 메뉴 이동

1. Supabase 프로젝트를 연다.
2. **Project Settings**로 이동한다.
3. **API Keys** 메뉴를 연다.
4. **Secret keys** 또는 레거시 **service_role** key를 찾는다.

새 Secret key:

sb_secret_...

레거시 service role key:

eyJhbGciOi...

기록:

SUPABASE_SECRET_KEY=

또는:

SUPABASE_SERVICE_ROLE_KEY=

체크:

- [] 서버용 key를 확인했다.
- [] 이 값을 채팅창에 공개하지 않는다.
- [] 이 값을 GitHub에 올리지 않는다.
- [] 이 값에는 **NEXT_PUBLIC_**을 붙이지 않는다.

7. Supabase Storage bucket 생성

7.1 메뉴 이동

1. Supabase 프로젝트를 연다.
2. 왼쪽 메뉴에서 **Storage**를 누른다.
3. **Buckets** 화면으로 이동한다.
4. **New bucket** 또는 **Create bucket**을 누른다.

7.2 bucket 이름 입력

추천:

phoenix-pages-images

환경변수 기록:

SUPABASE_STORAGE_BUCKET=phoenix-pages-images

7.3 Public / Private 선택

운영형 V1 권장:

Private bucket

빠른 테스트만 할 때:

Public bucket도 가능

하지만 유저 이미지가 들어가므로 Private을 권장한다.

체크:

- [] Storage bucket을 만들었다.
- [] bucket 이름을 기록했다.
- [] 운영형이면 Private bucket으로 만들었다.
- [] 이미지 파일 저장 목적임을 확인했다.

8. Google OAuth Client 생성

V1에서 Google 로그인을 사용하려면 Google Cloud에서 OAuth Client를 만들어야 한다.

8.1 Google Cloud Console 접속

1. 브라우저에서 Google Cloud Console에 접속한다.

<https://console.cloud.google.com/>

2. Google 계정으로 로그인한다.
3. 상단 프로젝트 선택 영역을 누른다.
4. 기존 프로젝트를 선택하거나 새 프로젝트를 만든다.

추천 프로젝트명:

phoenix-pages-auth

체크:

- [] Google Cloud 프로젝트를 선택했다.
- [] 프로젝트 이름을 확인했다.

9. Google Auth Platform 설정

9.1 Branding 설정

1. Google Cloud Console에서 **Google Auth Platform**을 연다.
2. **Branding** 메뉴를 연다.
3. App name을 입력한다.

추천:

Phoenix Pages

4. User support email을 선택한다.
5. Developer contact email을 입력한다.
6. 저장한다.

체크:

- [] App name을 입력했다.
- [] 지원 이메일을 입력했다.
- [] 개발자 연락 이메일을 입력했다.

9.2 Audience 설정

1. **Audience** 메뉴를 연다.
2. 테스트 단계라면 Test users에 마스터 이메일을 추가한다.
3. 필요하면 수강생 테스트 이메일도 추가한다.

체크:

- [] 테스트 유저에 마스터 이메일을 추가했다.
- [] 로그인 테스트에 사용할 이메일을 확인했다.

9.3 Scopes 확인

기본 로그인에 필요한 범위:

openid
email
profile

체크:

- [] 불필요한 민감 권한을 추가하지 않았다.
- [] 이메일과 프로필 기본 권한만 사용한다.

10. Supabase callback URL 확인

Google OAuth Client를 만들기 전에 Supabase callback URL을 알아야 한다.

10.1 메뉴 이동

1. Supabase 프로젝트를 연다.
2. **Authentication**을 누른다.
3. **Providers**를 연다.
4. **Google**을 클릭한다.
5. 화면에 표시된 Callback URL을 확인한다.

보통 이런 형식이다.

`https://xxxxxxxxxxxxxxxxxxxxx.supabase.co/auth/v1/callback`

체크:

- [] Supabase Google Provider 화면을 열었다.
- [] Callback URL을 복사했다.

11. Google OAuth Client 만들기

11.1 Clients 메뉴 이동

1. Google Cloud Console에서 **Google Auth Platform**을 연다.
2. **Clients** 메뉴를 연다.
3. **Create client** 또는 **Create OAuth client**를 누른다.

11.2 Application type 선택

선택:

Web application

이름:

Phoenix Pages Web

11.3 Authorized JavaScript origins 입력

로컬 개발:

`http://localhost:3002`

Vercel 배포:

`https://내-vercel-배포주소.vercel.app`

커스텀 도메인 사용 시:

`https://내-커스텀-도메인`

주의:

origin에는 /auth/callback 같은 경로를 넣지 않는다.
도메인까지만 넣는다.

11.4 Authorized redirect URIs 입력

Supabase에서 확인한 callback URL을 넣는다.

`https://xxxxxxxxxxxxxxxxxxxxx.supabase.co/auth/v1/callback`

체크:

- ☐ Web application으로 만들었다.
- ☐ JavaScript origin에 localhost를 넣었다.
- ☐ JavaScript origin에 Vercel URL을 넣었다.
- ☐ Redirect URI에 Supabase callback URL을 넣었다.
- ☐ Client ID를 복사했다.
- ☐ Client Secret을 복사했다.

기록:

GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=

12. Supabase Google Provider 활성화

12.1 메뉴 이동

1. Supabase 프로젝트를 연다.
2. **Authentication**으로 이동한다.
3. **Providers**를 연다.
4. **Google**을 클릭한다.

12.2 Client ID / Secret 입력

1. **Enable Sign in with Google**을 켜다.
2. Google Cloud에서 복사한 Client ID를 입력한다.
3. Google Cloud에서 복사한 Client Secret을 입력한다.
4. 저장한다.

체크:

- ☐ Google Provider가 Enabled 상태다.
- ☐ Client ID를 입력했다.

- [] Client Secret을 입력했다.
- [] 저장했다.

13. Supabase URL Configuration 설정

로그인 후 앱으로 돌아오기 위한 설정이다.

13.1 메뉴 이동

1. Supabase 프로젝트를 연다.
2. **Authentication**으로 이동한다.
3. **URL Configuration** 또는 **URL Config**를 연다.

13.2 Site URL 입력

Vercel 배포 URL:

`https://내-vercel-배포주소.vercel.app`

커스텀 도메인:

`https://내-커스텀-도메인`

13.3 Redirect URLs 입력

로컬 개발:

`http://localhost:3002/**`

Vercel 배포:

`https://내-vercel-배포주소.vercel.app/**`

커스텀 도메인:

`https://내-커스텀-도메인/**`

체크:

- [] Site URL을 입력했다.
- [] localhost Redirect URL을 입력했다.
- [] Vercel Redirect URL을 입력했다.
- [] 커스텀 도메인이 있으면 추가했다.
- [] 저장했다.

14. 슈퍼 관리자 이메일 결정

슈퍼 관리자는 전체 유저의 프로젝트를 볼 수 있는 마스터 계정이다.

14.1 이메일 기준

Google 로그인에 사용할 이메일과 정확히 같아야 한다.

예:

master@example.com

환경변수:

SUPER_ADMIN_EMAILS=master@example.com

여러 명이면:

SUPER_ADMIN_EMAILS=master@example.com,admin@example.com

체크:

- [] 슈퍼 관리자 이메일을 정했다.
- [] Google 로그인 이메일과 같은지 확인했다.
- [] 오타가 없는지 확인했다.

15. Vercel 환경변수 입력

코덱스가 V1 코드를 완성하면 Vercel에 환경변수를 입력해야 한다.

15.1 메뉴 이동

1. Vercel Dashboard에 접속한다.
2. **phoenix pages** 프로젝트를 선택한다.
3. **Settings**를 누른다.
4. **Environment Variables**를 연다.

15.2 입력할 환경변수

프로젝트 코드가 요구하는 이름에 맞춰 입력한다.

후보:

```
NEXT_PUBLIC_SUPABASE_URL=  
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=  
NEXT_PUBLIC_SUPABASE_ANON_KEY=  
SUPABASE_SECRET_KEY=  
SUPABASE_SERVICE_ROLE_KEY=  
SUPABASE_STORAGE_BUCKET=phoenix-pages-images  
GOOGLE_CLIENT_ID=  
GOOGLE_CLIENT_SECRET=
```

SUPER_ADMIN_EMAILS=
DATABASE_URL=

주의:

NEXT_PUBLIC_로 시작하는 값만 브라우저 공개 가능 값이다.
SECRET, SERVICE_ROLE, CLIENT_SECRET, DATABASE_URL은 절대 NEXT_PUBLIC_을 붙이지 않는다.

체크:

- [] Supabase URL을 입력했다.
- [] Supabase 공개용 key를 입력했다.
- [] Supabase 서버용 key를 입력했다.
- [] Storage bucket 이름을 입력했다.
- [] Google Client ID를 입력했다.
- [] Google Client Secret을 입력했다.
- [] Super Admin 이메일을 입력했다.
- [] Production, Preview, Development 환경에 필요한 범위를 확인했다.
- [] 환경변수 저장 후 재배포했다.

16. 코덱스가 제공한 SQL 실행

코덱스가 V1 작업을 완료하면 Supabase SQL을 제공해야 한다.

16.1 메뉴 이동

1. Supabase 프로젝트를 연다.
2. **SQL Editor**를 누른다.
3. **New query**를 누른다.
4. 코덱스가 제공한 SQL을 붙여넣는다.
5. **Run**을 누른다.

체크:

- [] SQL을 실행했다.
- [] 오류 없이 완료됐다.
- [] **profiles** 테이블이 생성됐다.
- [] **projects** 테이블이 생성됐다.
- [] **project_sections** 테이블이 생성됐다.
- [] **admin_users** 테이블이 생성됐다.
- [] RLS가 켜져 있다.
- [] 기본 정책이 생성됐다.

17. V1 로그인 테스트

17.1 로컬 테스트

1. **.env.local**에 필요한 환경변수를 넣는다.
2. 로컬 서버를 실행한다.

`npm run dev:local`

3. 브라우저에서 접속한다.

`http://localhost:3002`

4. Google 로그인 버튼을 누른다.
5. 로그인 후 앱으로 돌아오는지 확인한다.

체크:

- ☐ Google 로그인 버튼이 보인다.
- ☐ Google 계정 선택 화면이 열린다.
- ☐ 로그인 후 phoenix pages로 돌아온다.
- ☐ 유저 이메일이 표시된다.
- ☐ 로그아웃이 가능하다.

17.2 Vercel 테스트

1. Vercel 배포 URL에 접속한다.
2. Google 로그인 버튼을 누른다.
3. 로그인 후 배포 URL로 돌아오는지 확인한다.

체크:

- ☐ Vercel 배포 URL에서 로그인 가능하다.
- ☐ 로그인 후 redirect 오류가 없다.
- ☐ Site URL과 Redirect URL이 맞다.

18. V1 저장 테스트

18.1 일반 유저 저장

1. 일반 유저 Google 계정으로 로그인한다.
2. 이미지를 업로드한다.
3. 상세페이지 1장을 생성한다.

4. **결과 저장**을 누른다.
5. 작업 목록에서 저장된 프로젝트를 확인한다.
6. 새로고침 후 다시 열리는지 확인한다.

체크:

- [] 생성 이미지가 화면에 보인다.
- [] Storage에 이미지 파일이 저장됐다.
- [] Database에 project row가 생성됐다.
- [] Database에 section row가 생성됐다.
- [] 새로고침 후 프로젝트를 다시 열 수 있다.
- [] 다른 일반 유저 계정에서는 이 프로젝트가 보이지 않는다.

18.2 슈퍼 관리자 확인

1. 슈퍼 관리자 이메일로 로그인한다.
2. 관리자 모드 또는 관리자 작업 목록을 연다.
3. 일반 유저가 만든 프로젝트가 보이는지 확인한다.

체크:

- [] 슈퍼 관리자 계정으로 로그인했다.
- [] 전체 유저 작업 목록이 보인다.
- [] 유저 이메일 또는 user id를 확인할 수 있다.
- [] 일반 유저 계정에서는 전체 작업 목록이 보이지 않는다.

19. V0 fallback 테스트

V1이 되어도 V0 기능은 유지되어야 한다.

19.1 비로그인 상태 테스트

1. 로그아웃한다.
2. 이미지를 업로드한다.
3. 1장 생성한다.
4. 결과 다운로드를 누른다.
5. 브라우저 최근 작업이 보이는지 확인한다.

체크:

- [] 비로그인 상태에서도 생성 가능하다.
- [] 비로그인 상태에서도 ZIP 다운로드 가능하다.

- ☐ 비로그인 상태에서는 클라우드 저장 대신 로그인 안내가 나온다.
- ☐ 기존 V0 기능이 깨지지 않았다.

20. 이미지 용량 문제 확인

V1의 중요한 목적 중 하나는 이미지 데이터를 브라우저와 서버가 통째로 주고받는 문제를 줄이는 것이다.

20.1 테스트

1. 로그인한다.
2. 9:16 상세페이지 1장을 생성한다.
3. 결과 저장을 누른다.
4. Storage에 이미지가 저장됐는지 확인한다.
5. 저장된 프로젝트를 다시 연다.
6. 개별 편집을 실행한다.

체크:

- ☐ 이미지가 base64 data URL만으로 저장되지 않는다.
- ☐ Storage path 또는 URL이 DB에 저장된다.
- ☐ 편집 요청에서 이미지 전체 데이터 전송이 줄어든다.
- ☐ **이미지 데이터가 너무 커서...** 오류가 줄어든다.

주의:

V1이 되어도 외부 이미지 생성 API 자체의 timeout이나 실패는 0이 되지 않는다.
하지만 브라우저 저장과 요청 용량 문제는 크게 완화된다.

21. 마스터 최종 확인표

아래 항목이 모두 체크되면 V1 전환이 완료된 것이다.

- ☐ Supabase 프로젝트 준비 완료
- ☐ Supabase Storage bucket 생성 완료
- ☐ Google OAuth Client 생성 완료
- ☐ Supabase Google Provider 활성화 완료
- ☐ Supabase URL Configuration 설정 완료
- ☐ Vercel 환경변수 입력 완료
- ☐ 코덱스 제공 SQL 실행 완료
- ☐ **npm run build** 성공
- ☐ 로컬 Google 로그인 성공

- [] Vercel Google 로그인 성공
- [] 일반 유저 프로젝트 저장 성공
- [] 일반 유저 프로젝트 다시 열기 성공
- [] 슈퍼 관리자 전체 작업 조회 성공
- [] 일반 유저는 자기 작업만 조회
- [] Storage에 이미지 파일 저장 확인
- [] DB에 프로젝트와 섹션 저장 확인
- [] V0 비로그인 fallback 유지
- [] ZIP 다운로드 유지
- [] 개별 편집 유지
- [] AI 멘트 생성 유지

22. 문제가 생겼을 때 확인 순서

22.1 Google 로그인 후 돌아오지 않음

확인:

1. Supabase Auth **Site URL**
2. Supabase Auth **Redirect URLs**
3. Google OAuth **Authorized redirect URIs**
4. Vercel 실제 배포 URL

22.2 Storage 저장 실패

확인:

1. bucket 이름이 환경변수와 같은가
2. 서버용 secret/service role key가 설정됐는가
3. API route가 server에서 실행되는가
4. 파일 크기 제한에 걸리지 않았는가
5. Storage policy가 막고 있지 않은가

22.3 일반 유저가 다른 유저 작업을 봄

즉시 확인:

1. RLS가 켜져 있는가
2. policy가 **auth.uid()** 기준으로 작성됐는가
3. service role key를 client에서 쓰고 있지 않은가
4. 관리자 조회 API가 일반 유저에게 열려 있지 않은가

22.4 슈퍼 관리자 모드가 안 열림

확인:

1. 로그인 이메일과 **SUPER_ADMIN_EMAILS**가 정확히 일치하는가
2. 실패 뒤 공백 처리 문제가 없는가
3. admin_users 테이블을 쓰는 경우 해당 이메일이 등록됐는가
4. 서버와 클라이언트가 최신 배포인지 확인

22.5 V0 기능이 깨짐

확인:

1. 비로그인 상태에서 생성 가능한가
2. API 키 설정이 그대로 작동하는가
3. 결과 다운로드가 ZIP으로 되는가
4. 브라우저 최근 작업이 남는가

23. 코덱스에게 다시 요청할 때 사용할 문장

문제가 생기면 아래처럼 구체적으로 요청한다.

V1 작업 후 Google 로그인은 되지만 Storage 저장에 실패합니다.
Supabase bucket 이름은 phoenix-pages-images이고,
Vercel 환경변수 SUPABASE_STORAGE_BUCKET도 같은 값입니다.
Storage 저장 API와 Supabase admin client 설정을 확인하고 수정해주세요.
기존 V0 기능은 유지해주세요.

또는:

V1 작업 후 일반 사용자가 다른 유저 프로젝트를 볼 수 있습니다.
RLS 정책과 projects 조회 API를 확인해서 일반 유저는 자기 프로젝트만 보이게 수정해주세요.
슈퍼 관리자만 전체 프로젝트를 볼 수 있어야 합니다.

24. 마스터 운영 원칙

1. 실제 secret key는 수강생 화면 공유나 녹화에 노출하지 않는다.
2. 수강생에게는 각자 Supabase 프로젝트를 만들게 한다.
3. 수강생이 만든 프로젝트의 secret key를 마스터가 수집하지 않는다.
4. 수강생 GitHub 저장소에 **.env.local**이 올라가지 않게 확인한다.
5. V1 작업 후에도 V0 fallback이 남아 있는지 확인한다.
6. V1.5 결제 기능은 V1 안정화 후 진행한다.
7. V2 봇 자동화는 V1 저장 구조가 안정된 뒤 진행한다.

25. MCP 보조 방식으로 진행할 때 마스터 체크리스트

Supabase MCP를 사용하면 코덱스가 Supabase 프로젝트 상태를 직접 확인하고 SQL, RLS, Storage 관련 작업을 더 빠르게 도울 수 있다.

하지만 마스터는 연결 범위와 권한을 반드시 확인해야 한다.

25.1 Supabase MCP 공식 정보

공식 문서:

<https://supabase.com/docs/guides/ai-tools/mcp>

공식 GitHub:

<https://github.com/supabase/mcp>

remote endpoint:

<https://mcp.supabase.com/mcp>

프로젝트 제한 형식:

https://mcp.supabase.com/mcp?project_ref=YOUR_PROJECT_REF

초기 확인용 권장 형식:

https://mcp.supabase.com/mcp?project_ref=YOUR_PROJECT_REF&read_only=true&features=database,docs,storage

체크:

- [] Supabase MCP 공식 문서를 확인했다.
- [] MCP 연결 대상이 내 Supabase 프로젝트인지 확인했다.
- [] **project_ref**를 확인했다.
- [] 처음에는 **read_only=true**로 연결했다.
- [] Storage 확인이 필요하면 **features=database,docs,storage**를 포함했다.

26. MCP 연결 전 확인할 것

MCP를 연결하기 전에 아래 항목을 확인한다.

- [] 수강생 개인 Supabase 프로젝트를 사용한다.
- [] 마스터의 운영 프로젝트를 수강생 MCP에 연결하지 않는다.
- [] 다른 수강생 프로젝트를 연결하지 않는다.
- [] read-only 상태에서 먼저 확인한다.
- [] SQL 실행 또는 migration 적용은 마스터 승인 후 진행한다.

- [] secret key를 코덱스 채팅창에 붙여넣지 않는다.
- [] MCP 로그나 화면 공유에 민감값이 노출되지 않는지 확인한다.

27. MCP로 코덱스가 확인하면 좋은 항목

코덱스에게 아래 항목을 확인하게 한다.

1. Supabase 프로젝트 연결 상태
2. Project URL
3. 공개용 key 종류
4. Storage bucket 목록
5. phoenix-pages-images bucket 존재 여부
6. 기존 Database table 목록
7. RLS 활성화 여부
8. 기존 policy 목록
9. SQL 실행 권한 여부
10. 최근 오류 로그

마스터 체크:

- [] 코덱스가 프로젝트 연결 상태를 보고했다.
- [] 코덱스가 Storage bucket 존재 여부를 보고했다.
- [] 코덱스가 기존 table 목록을 보고했다.
- [] 코덱스가 RLS 상태를 보고했다.
- [] 코덱스가 쓰기 작업 전 승인을 요청했다.

28. MCP 쓰기 작업 승인 기준

아래 작업은 read-only로 할 수 없다.

마스터가 명시적으로 승인해야 한다.

1. SQL 실행
2. table 생성
3. table 삭제
4. column 변경
5. RLS policy 생성/수정/삭제
6. Storage bucket 생성/수정
7. Edge Function 배포
8. migration 적용

승인 전 코덱스에게 요구할 보고:

실행할 SQL:

생성할 테이블:

수정할 정책:

삭제되는 항목 여부:

rollback 가능 여부:
영향받는 기능:

체크:

- [] SQL 실행 전 내용을 확인했다.
- [] 삭제 작업이 없는지 확인했다.
- [] 기존 V0 기능에 영향이 없는지 확인했다.
- [] 승인 후 쓰기 작업을 진행했다.

29. Google OAuth는 MCP 보조보다 수동 확인 권장

Google Cloud에도 MCP/gcloud 기반 도구가 있을 수 있다.

하지만 V1의 Google 로그인 설정은 아래 민감 작업을 포함한다.

Google OAuth Client 생성
Client Secret 발급
Authorized JavaScript origins 입력
Authorized redirect URIs 입력
Supabase Google Provider에 Client ID / Secret 입력

따라서 수강생용 권장 방식:

Google OAuth Client 생성 = 마스터가 Google Cloud Console에서 직접
Supabase Google Provider 활성화 = 마스터가 Supabase Dashboard에서 직접
코덱스 역할 = redirect URL 안내, 코드 연동, 오류 분석

체크:

- [] Google OAuth Client는 마스터가 직접 만들었다.
- [] Client Secret은 채팅창에 붙여넣지 않았다.
- [] Supabase Google Provider에는 마스터가 직접 입력했다.
- [] Google redirect URI와 Supabase callback URL이 일치한다.
- [] Vercel URL과 Supabase Redirect URLs가 일치한다.

30. MCP 방식으로 진행할 때 코덱스에게 보낼 문장

아래 문장을 코덱스에게 보낸다.

Supabase MCP를 사용할 수 있다면 먼저 read-only로 현재 프로젝트를 확인해주세요.

확인할 항목:

- project_ref
- Project URL
- Storage bucket 목록
- phoenix-pages-images bucket 존재 여부
- Database table 목록
- RLS 상태
- 기존 policy

- 오류 로그

SQL 실행, RLS 수정, Storage bucket 생성 같은 쓰기 작업은 먼저 보고하고 제 승인을 받은 뒤 진행해주세요.

Google OAuth Client 생성과 Supabase Google Provider Client Secret 입력은 제가 직접 하겠습니다.
코드에서는 Supabase Auth Google login이 동작하도록 구현해주세요.